Atlas — Technical Product Requirements Document

Full Technical Specification for a Modular, Multi-Branch Store Operations Platform
Version 1.0 · 2026-09-18 · Status: Engineering Baseline

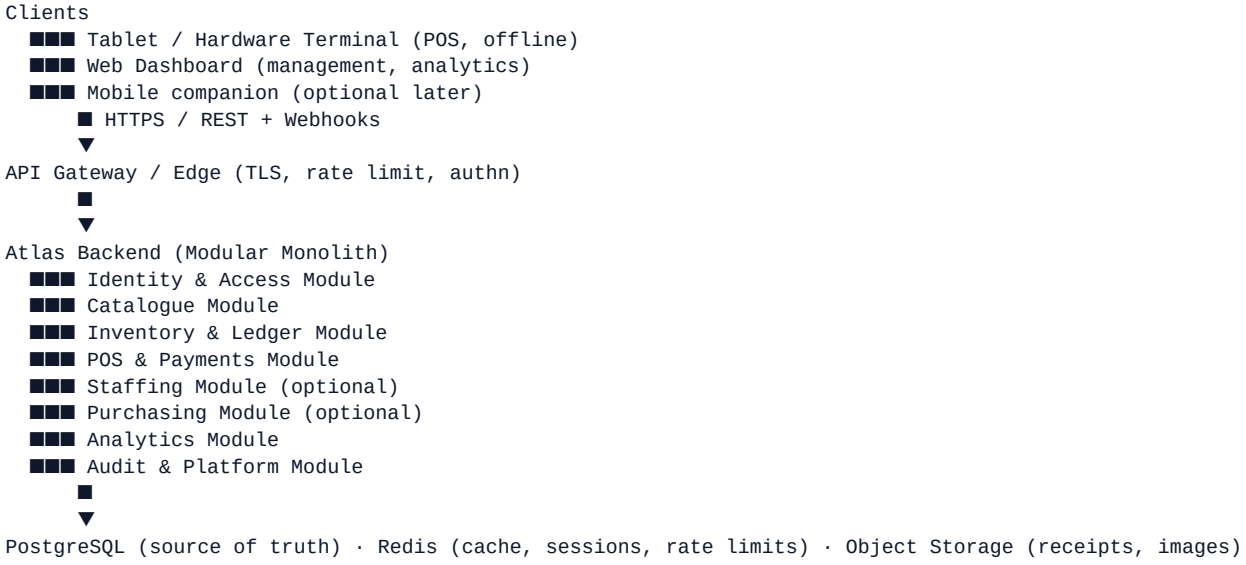
1. Overview & Goals

This Tech PRD defines the technical shape of Atlas: architecture, data model, module boundaries, API surface, security, concurrency, offline strategy, observability, and delivery constraints. It implements the organizational and modular rules described in the Systems Requirements document and supports the product vision of a single ledger for POS, inventory, staffing, and analytics across one or many stores.

Primary technical goals: (1) Strong tenant and store isolation, (2) Modular feature enablement, (3) Transactional inventory and payments, (4) Offline-capable POS with idempotent sync, (5) Horizontal scalability of the API tier without premature microservices, (6) Clear extension points for hardware, payment providers, and future modules.

2. High-Level Architecture

Atlas is delivered as a **modular monolith**. A single deployable backend hosts clearly separated modules that communicate via in-process interfaces. The database is shared (with tenant_id + store_id on every row). This avoids distributed transaction complexity in V1 while still allowing modules to be extracted later if needed.



2.1 Recommended Technology Stack

Layer	Choice	Rationale
Language / Runtime	TypeScript (Node 20+) or Go 1.22+	Strong typing, fast iteration, excellent concurrency; either is acceptable
API Framework	NestJS (TS) or Chi/Echo (Go) or Axum (Rust)	Modular structure, DI, guards, OpenAPI
Database	PostgreSQL 16+	JSONB for product attributes, strong constraints, row locking, mature tooling
ORM / Query	Prisma / Drizzle (TS) or sqlx / ent (Go) + raw SQL for hot paths	Migrations, type safety, escape hatch for performance
Cache / Sessions	Redis 7+	Sessions, rate limits, short-lived locks, product cache

Layer	Choice	Rationale
Auth	JWT (access) + rotating refresh + server-side session blacklist	Stateless access + controllable revocation
Password	Argon2id	Modern, memory-hard KDF
Payments	Provider abstraction (Paystack / Flutterwave / Stripe)	Swap or multi-provider without rewriting core
Object Storage	S3-compatible (R2, S3, MinIO)	Receipts, product images
Observability	OpenTelemetry + structured JSON logs (pino/zap)	Traces, metrics, request correlation
Container	Docker + docker-compose; later K8s	Reproducible deploys
CI	GitHub Actions / equivalent	Test, migrate, build, scan

Note: The Systems Requirements and product goals are stack-agnostic. The choices above optimize for modularity, developer velocity, and operational maturity. A Rust (Axum + SQLx) variant is also viable if the team prioritizes maximum performance and memory safety from day one.

3. Tenancy, Data Model & Isolation

Every business table carries **tenant_id**. Operational tables that are store-scoped also carry **store_id**. A global query filter (or PostgreSQL RLS) injects tenant_id from the authenticated principal. Store scope is enforced in the application authorization layer after the tenant filter has run.

3.1 Core Entities (simplified)

```
Tenant (id, name, status, default_currency, settings jsonb)
Store (id, tenant_id, code, name, address, timezone, currency, tax_profile_id, status, settings jsonb)
User (id, email, password_hash, name, status, ...)
Membership (user_id, tenant_id, status) — user belongs to tenant
RoleAssignment (user_id, role, store_id nullable, tenant_id) — null store_id = tenant-wide
ProductType (id, tenant_id, name, attribute_schema jsonb, identity_mode)
Product (id, tenant_id, product_type_id, sku, name, base_unit, attributes jsonb, status)
ProductStore (product_id, store_id, price, cost, low_stock_threshold, active)
StockBalance (store_id, product_id, batch_id?, qty_on_hand, qty_reserved)
StockMovement (id, tenant_id, store_id, product_id, type, qty, reason, actor_id, ref_type, ref_id,
created_at)
Sale / SaleLine / Payment / Shift / ... (all carry tenant_id + store_id)
```

3.2 Isolation Requirements

- Tenant_id is derived exclusively from the authenticated principal (or from an internal service account). Never from query params, body, or headers controlled by the client.
- Composite indexes begin with tenant_id (and store_id where relevant).
- Integration tests must prove that a query written without the tenant filter cannot return another tenant's data.
- Cross-store reads are allowed only when the caller holds a tenant-scoped role or an explicit multi-store grant.

4. Module Boundaries & Contracts

Each module owns its tables, domain services, and HTTP routes. Modules depend on each other only through explicit application-service interfaces (no reaching into another module's repository). Module enablement is checked by a lightweight guard before route handlers run.

Module	Owns	Key Interfaces
Identity	Tenants, Stores, Users, Roles, Sessions	AuthN, AuthZ guard, CurrentUser, StoreContext
Catalogue	ProductTypes, Products, Prices, Units	ResolvePrice, GetProduct, SearchProducts
Inventory	Balances, Movements, Reservations, Transfers	Reserve, Commit, Adjust, Transfer, GetAvailable

Module	Owns	Key Interfaces
POS	Sales, Lines, Payments, Shifts, Receipts	CreateSale, Void, Return, Open/CloseShift
Staffing	Schedules, Clock events, Labor rules	CreateShift, ClockIn/Out, LaborCost
Purchasing	Suppliers, POs, GRNs	ReceiveStock → Inventory
Analytics	Materialized views / read models	SalesSummary, Valuation, LowStock, Comparison
Audit	AuditLog	Record(actor, action, resource, outcome)
Platform	Feature flags, Entitlements, Health	IsModuleEnabled, Health, Ready

5. Critical Business Flows (Technical)

5.1 Checkout (Online)

1. Authenticate + resolve store context and permissions.
2. Validate cart lines against Catalogue (existence, active, price resolution).
3. Begin DB transaction.
4. For each line: Inventory.Reserve(store, product, qty) under row lock; fail if insufficient.
5. Create Sale + SaleLines (historical price/tax snapshot) + Payment intent.
6. Commit.
7. Initialize external payment (if card/transfer); on success webhook → confirm payment, convert reservation to consumption.
8. Emit receipt; update shift totals if applicable.

5.2 Offline Checkout & Sync

• Client generates UUIDv7 (or ULID) for sale_id and line_ids; attaches device_id.
• Sale is stored in a durable local queue (SQLite / IndexedDB) with full payload.
• On reconnect, POST /api/v1/sync/sales with the batch. Server upserts by (tenant_id, client_sale_id) — idempotent.
• Inventory is adjusted only on successful server accept; conflicts (e.g. stock gone) return structured errors for client resolution.
• Device_id and client timestamps are retained for audit.

5.3 Inter-Store Transfer

Tenant Admin (or granted role) creates a TransferOrder (source_store, dest_store, lines). On ship: Inventory movement type=TRANSFER_OUT at source. On receive: TRANSFER_IN at destination. Both sides are linked by transfer_id; partial receives are supported. Total tenant stock is conserved.

5.4 Authorization Decision

For every request:
1. Authenticate → principal (user_id, tenant_id).
2. Resolve requested store_id (from path, body, or session default).
3. Load RoleAssignments for (user, tenant).
4. Evaluate required permission against assignments that cover the store (or tenant-wide).
5. Allow or 403. Never rely on UI alone.

6. MVP API Surface (Illustrative)

Base: /api/v1. All routes except auth and health require a valid access token. Store-scoped routes accept X-Store-Id or a store_id path/query parameter that is then authorized.

Area	Methods	Notes
Auth	POST /auth/register, /login, /refresh, /logout, /forgot, /reset	Rate-limited
Me / Users	GET /me, PATCH /me; Admin: CRUD users & role assignments	Tenant Admin for writes
Stores	GET/POST /stores, GET/PATCH /stores/:id	Create = Tenant Admin
Products	GET /products, GET /products/:id, POST/PATCH (admin)	Search, filter, pagination
Inventory	GET balances, POST adjustments, POST transfers, GET movements	Store-scoped
Cart / Sales	POS session, POST /sales, GET /sales, POST /sales/:id/void|return	Idempotency-Key supported
Payments	POST /payments/initialize, webhook endpoint, GET status	Provider-agnostic
Shifts	POST open/close, GET current	Cashier + Manager
Reports	GET /reports/sales, /inventory/valuation, /low-stock, /comparison	Scope by role
Admin / Platform	GET /health, /ready; module entitlements	Internal + Tenant Admin

7. Error Handling

Consistent envelope:

```
{ "error": { "code": "PRODUCT_OUT_OF_STOCK", "message": "…", "details": { … } } }
```

Standard codes include: VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT, PRODUCT_OUT_OF_STOCK, PAYMENT_FAILED, MODULE_DISABLED, IDEMPOTENCY_CONFLICT, INTERNAL_ERROR.

8. Security & Compliance

- TLS everywhere in production; HSTS.
- Argon2id password hashing; no plaintext or reversible storage.
- Short-lived access tokens (5–15 min); rotating refresh tokens; server-side revocation list for logout / compromise.
- PCI: prefer payment-provider hosted fields / redirects so card data never touches Atlas servers. If present, isolate and minimize scope.
- Secrets in environment / secret manager; never in git.
- Parameterized queries only; input validation on every boundary (validator / zod / equivalent).
- Rate limiting on auth and sensitive mutations.
- Audit log for login, role changes, stock adjustments, voids, refunds, module changes.

9. Observability, Health & Operations

- Structured JSON logs with request_id, tenant_id, store_id, user_id (where safe).
- Metrics: request latency, error rate, checkout success, inventory reservation conflicts, sync lag.
- Tracing (OpenTelemetry) across API → DB and payment calls.
- GET /health — process up.
- GET /ready — PostgreSQL + Redis reachable; critical migrations applied.
- Background jobs (queue): expired reservation cleanup, report materialization, email/receipt dispatch, webhook retries.

10. Testing Strategy

- Unit: price resolution, discount/tax math, order state transitions, available-stock calculation, permission evaluation.
- Integration: API → DB with real PostgreSQL (testcontainers); tenant isolation tests; concurrent reservation tests.
- Contract: OpenAPI validation; webhook signature verification.
- E2E: register → create store → add product → open shift → sell → pay → stock decreased → report reflects sale.
- Multi-branch E2E: Tenant Admin creates two stores, sells in both, verifies isolation for Store Manager and roll-up for Admin.
- Load: checkout path under concurrent load; prove no oversell.

11. Technical Delivery Phases

Phase	Focus	Exit Criteria
P0 Foundation	Project skeleton, config, DB, migrations, auth, tenancy filter, audit, health	Login works; tenant isolation proven by test
P1 Catalogue + Inventory	Products, store pricing, stock ledger, adjustments, low-stock	Cannot oversell under concurrency test
P2 POS + Payments	Sale flow, shifts, provider abstraction, webhooks, offline sync design	End-to-end paid sale + receipt
P3 Multi-Store + Roles	Store CRUD, role assignments, cross-store reports, transfers	Admin sees all; Manager scoped correctly
P4 Modules	Staffing, Purchasing toggles, feature flags, entitlements	Disabled module invisible and non-writable
P5 Hardening	Observability, load test, security review, backup/restore drill	Ready for pilot tenants

12. Guiding Engineering Principles

- Backend is the source of truth for price, stock, and payment status.
- Tenant and store isolation are non-negotiable; tested, not assumed.
- Financial and stock operations are transactional and idempotent.
- Modules are real boundaries; feature flags control presence, not just UI.
- Prefer a modular monolith until a concrete scaling or team boundary forces extraction.
- Make invalid states hard to represent (types, constraints, state machines).
- Delete complexity before adding features.
- Ship the narrowest vertical that proves the multi-branch ledger works, then expand.

This Tech PRD is the engineering baseline. It is subordinate to the Systems Requirements for organizational and modularity rules, and to the Product PRD for user-facing scope and prioritization. Major architectural changes require an update to this document and review with stakeholders.